

Apache Paimon Python

核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-python (pypaimon)

版本：release-1.3.1 | pypaimon 1.5.dev

生成工具：AI Architecture Analyzer

生成日期：2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-python (pypaimon) 是 Apache Paimon 湖仓一体数据管理平台的纯 Python SDK 实现。该模块使 Python 生态用户能够直接通过 Python API 访问 Paimon 表数据，无需依赖 JVM 环境。版本为 1.5.dev，支持 Python 3.6+，提供完整的表读写、Schema 管理、快照管理、分支/标签管理、CLI 工具等功能，并可与 Ray、PyTorch、Lance 等框架集成。

1.2 核心能力

- 双 Catalog 架构：FileSystemCatalog（文件系统元数据）+ RESTCatalog（REST API 驱动，版本管理）
- 多格式读取：Parquet、ORC、Avro、Lance、Blob 五种文件格式
- 多模式写入：AppendOnly / KeyValue 两种模式，支持 Overwrite/Truncate
- 文件系统抽象：FileIO 支持 Local、OSS、S3、HDFS 等
- 快照与版本：Snapshot/Tag/Branch 管理，Time Travel
- 数据演进：Schema Evolution + Data Evolution（跨 Schema 合并读取）
- 全局索引：BTree 索引读取器，向量搜索
- CLI 工具：paimon 命令行，catalog/db/table 操作
- AI/ML 集成：Ray DataSource/Sink、PyTorch Dataset、Lance 格式

1.3 源码规模

指标	数量
Python 源文件	342
测试文件	80
总文件数	405
核心子包数	20+

第2章 核心架构设计

2.1 五层分层架构

pypaimon 采用五层架构：CLI/入口层 → Catalog 层 → Table 层 → 读写引擎层 → 基础设施层。各层通过 ABC 抽象接口解耦。

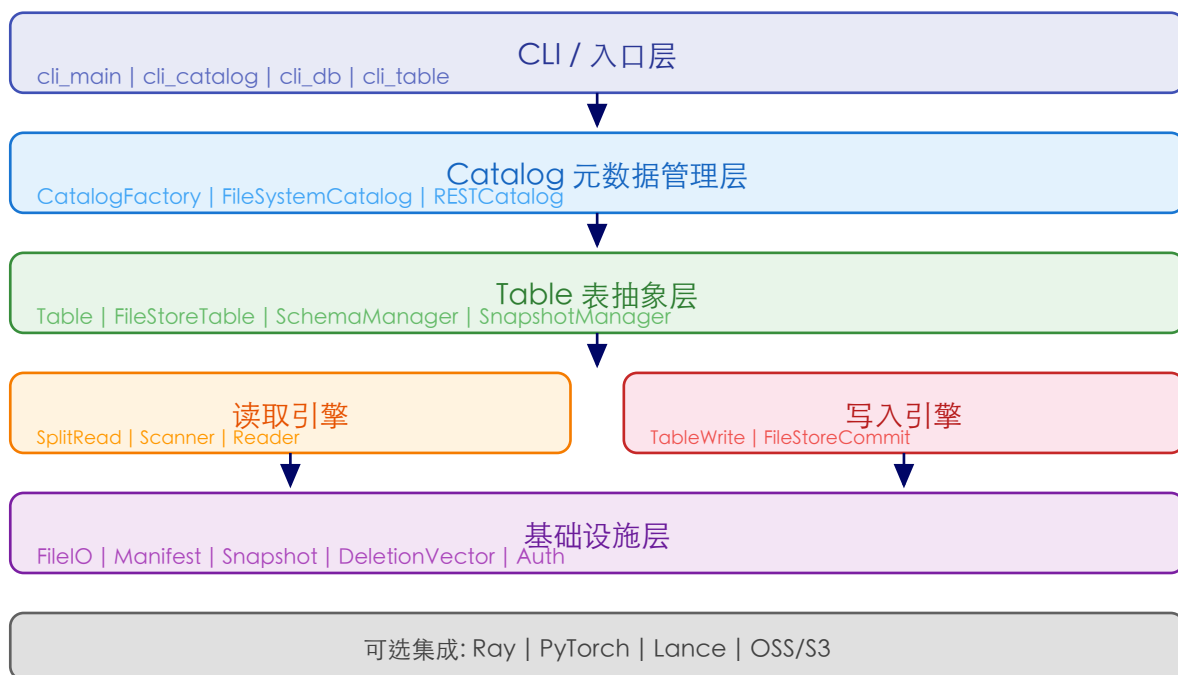


图2-1 pypaimon 五层分层架构图

2.2 核心子包清单

子包	核心文件	职责
catalog/	catalog.py, catalog_factory.py, filesystem_catalog.py, rest/rest_catalog.py	Catalog 抽象与实现
api/	rest_api.py, client.py, auth/	REST API 客户端、认证
table/	table.py, file_store_table.py	Table 抽象与核心实现
read/	split_read.py, reader/ (14+ Reader)	数据读取引擎
write/	file_store_commit.py, table_write.py	数据写入引擎
schema/	data_types.py, schema_manager.py	数据类型系统(25+种)
snapshot/	snapshot.py, snapshot_manager.py	快照管理
manifest/	manifest_file_manager.py, manifest_list_manager.py	Manifest 文件读写
filesystem/	pvfs.py (38KB), pyarrow_file_io.py	文件系统抽象
globalindex/	btree_index_reader.py	全局BTree索引
deletionvectors/	deletion_vector.py	删除向量(Bitmap)
branch/	filesystem_branch_manager.py, catalog_branch_manager.py	分支管理
tag/	tag.py, tag_manager.py	标签管理
cli/	cli_main.py, cli_table.py	命令行工具
integrations/	ray_source.py, ray_sink.py, paimon_dataset.py	Ray/PyTorch集成

2.3 类继承体系

Catalog 层：

```
Catalog (ABC) - list_databases, get_table, create_table, commit_snapshot
+-- FileSystemCatalog # 
+-- RESTCatalog # REST API
CatalogFactory # : filesystem / rest
```

Table 层：

```
Table (ABC) - new_read_builder, new_batch_write_builder
+-- FileStoreTable # / , BucketMode
+-- FormatTable # 
+-- IcebergTable # Iceberg
```

读取引擎：

```
SplitRead (ABC) - create_reader, file_reader_supplier
+-- RawFileSplitRead # : + DV
+-- MergeFileSplitRead # : KV + SortMerge
+-- DataEvolutionSplitRead # : Schema
```

FileIO 层：

```
FileIO (ABC) - new_input_stream, exists, delete, rename
+-- LocalFileIO # 
+-- PyArrowFileIO # (OSS/S3/HDFS)
+-- RESTTokenFileIO # REST Token ( + )
```

第3章 核心类设计模式详解

3.1 CatalogFactory — 工厂模式 (Factory Pattern)

CatalogFactory 通过 CATALOG_REGISTRY 字典维护类型到实现类的映射。用户调用 create(options) 根据 metastore 配置自动选择 FileSystemCatalog 或 RESTCatalog。新增 Catalog 类型仅需注册即可，符合开闭原则。

3.2 DefaultErrorHandler — 单例模式 (Singleton Pattern)

通过重写 __new__ 和 get_instance 实现单例，确保全局唯一的错误处理器处理所有 REST API 错误响应，按 HTTP 状态码映射到 BadRequest/NotAuthorized/Forbidden/NotFound/AlreadyExists/ServiceFailure 等异常。

3.3 Catalog / Table — 模板方法模式 (Template Method)

Catalog ABC 定义完整操作模板 (list_databases、get_table、create_table、commit_snapshot)，子类实现具体逻辑。部分方法如 rename_table、rollback_to 提供默认实现并抛出 NotImplementedError，子类选择性覆盖。

3.4 RESTTokenFileIO — 代理模式 (Proxy Pattern)

作为 FileIO 的代理，在文件操作前自动处理 REST Token 的获取、缓存和刷新。所有文件系统操作委托给内部 PyArrowFileIO，代理层仅负责认证逻辑，对调用者透明。

3.5 SplitRead 体系 — 策略模式 (Strategy Pattern)

SplitRead ABC 定义 create_reader() 接口，三种策略：RawFileSplitRead（非主键表直接读取+DV 过滤）、MergeFileSplitRead（主键表KV+SortMerge合并去重）、DataEvolutionSplitRead（跨Schema合并读取）。按表类型动态选择。

3.6 ConfigOptions — 建造者模式 (Builder Pattern)

ConfigOption 使用链式 Builder 构建配置定义：key().string_type().no_default_value().with_description()，使配置代码简洁可读。

3.7 AuthProviderFactory — 抽象工厂模式

根据配置动态创建 BearerTokenAuthProvider 或 DLFAuthProvider。DLF 进一步用策略模式区分 VPC (HMAC-SHA256) 和公网 (HMAC-SHA1) 签名。

3.8 FileStoreCommit — 乐观锁 + 重试模式

`_try_commit()` 实现乐观锁提交：准备 Manifest/Snapshot → 原子提交 → 失败时冲突检测 → 指数退避+抖动重试，最大重试次数和超时可配。包含 `ConflictDetection` 和 `CommitRollback` 机制。

第4章 核心流程图

4.1 表数据读取流程

用户通过 `ReadBuilder` 构建读取计划，`TableScan` 扫描 `Manifest` 获取 `Split`，`SplitRead` 按表类型选择策略读取数据文件，返回 `PyArrow Table`。

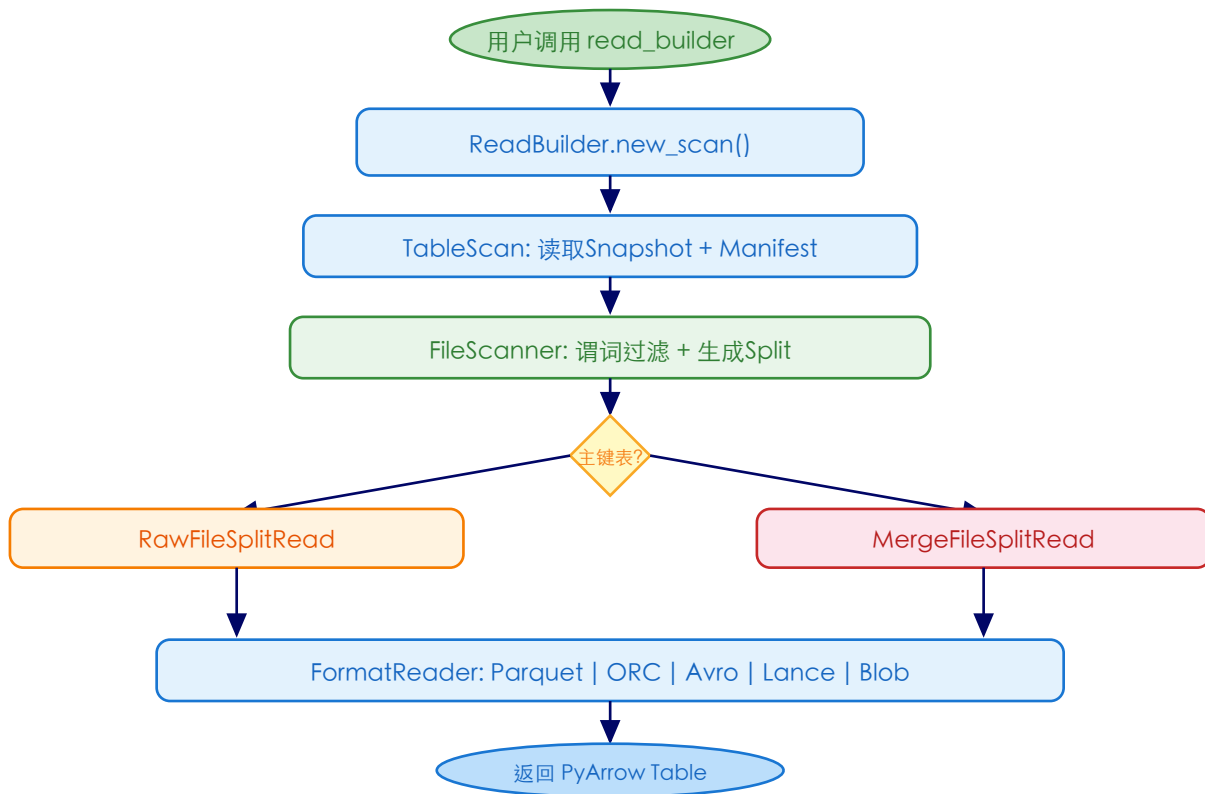


图 4-1 表数据读取核心流程图

4.2 数据写入与提交流程

写入阶段 TableWrite 按分区+Bucket 写入文件，提交阶段 FileStoreCommit 创建 Manifest/Snapshot 并原子提交。

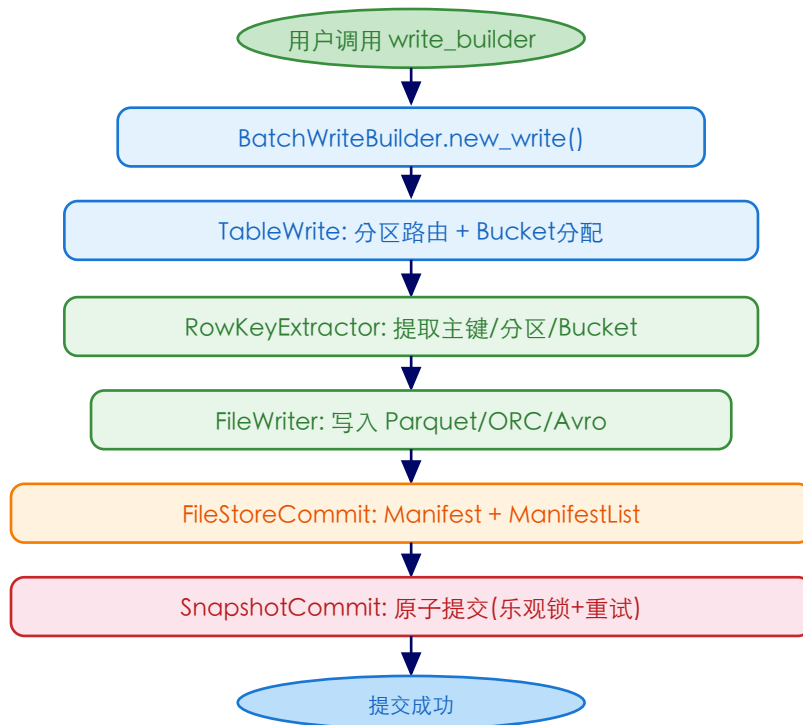


图4-2 数据写入与提交流程图

4.3 REST Catalog 初始化流程

RESTCatalog 初始化涉及认证提供者创建、HttpClient 配置、Token 获取和服务端配置同步。

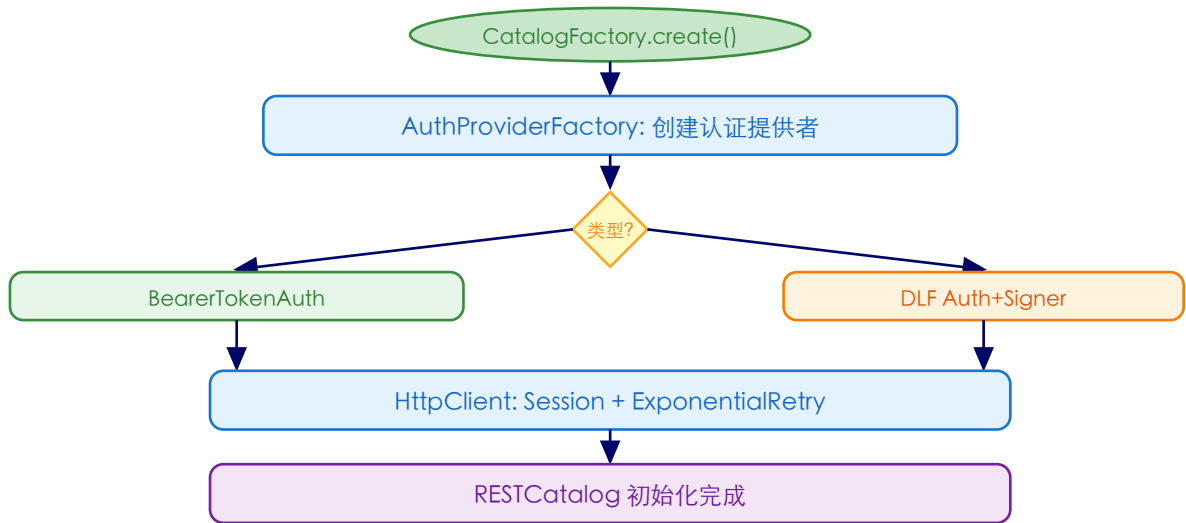


图4-3 REST Catalog 初始化流程图

第5章 核心时序图

5.1 批量读取时序图

用户通过 CatalogFactory 获取 Table 后执行批量读取的完整交互，涉及 Catalog、Table、ReadBuilder、TableScan、SplitRead。

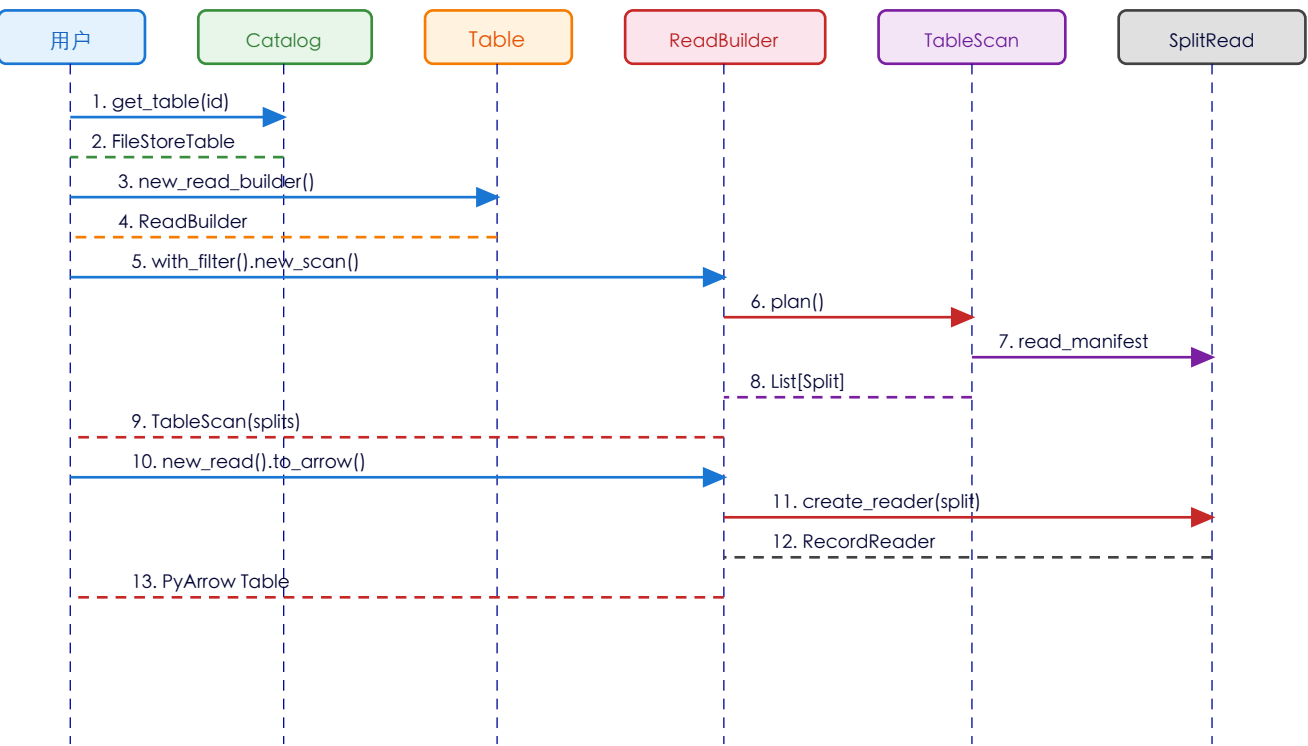


图5-1 批量读取完整时序图

5.2 写入提交时序图

数据写入和快照提交时序：TableWrite → FileWriter → FileStoreCommit → SnapshotCommit。

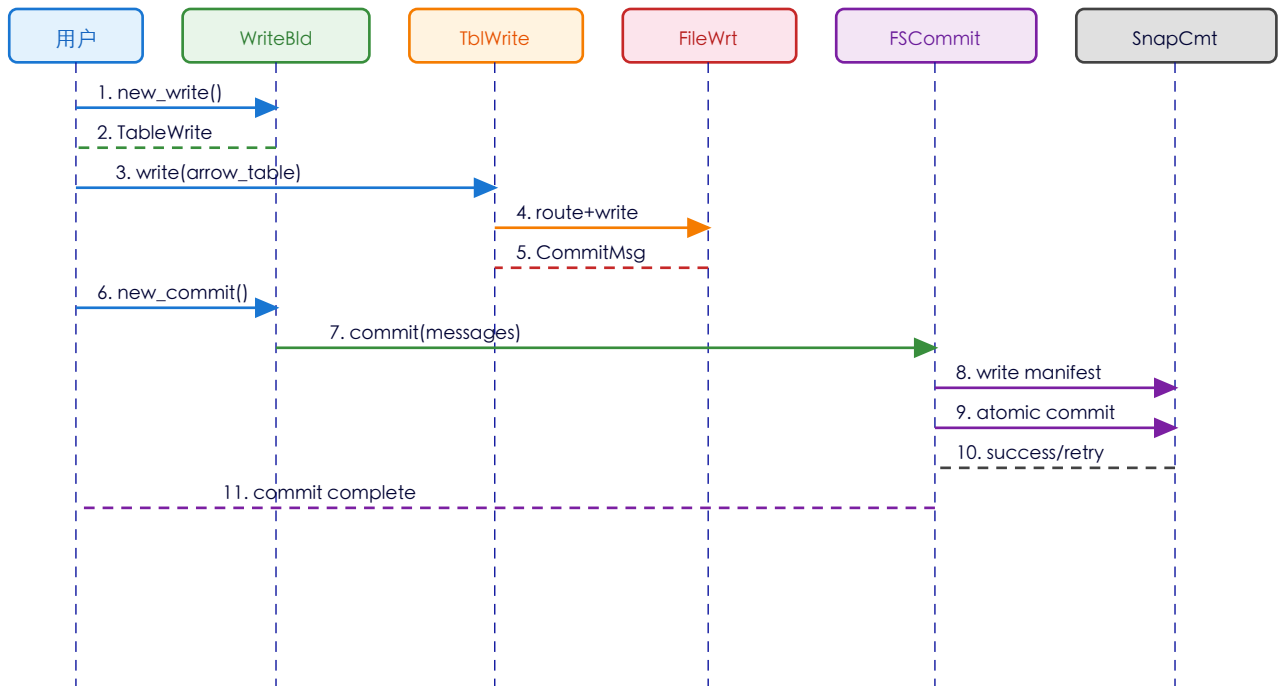


图5-2 写入提交时序图

5.3 REST Catalog 加载表时序图

RESTCatalog 通过 REST API 加载表：认证 → HTTP 请求 → 解析响应 → 构建 FileStoreTable。

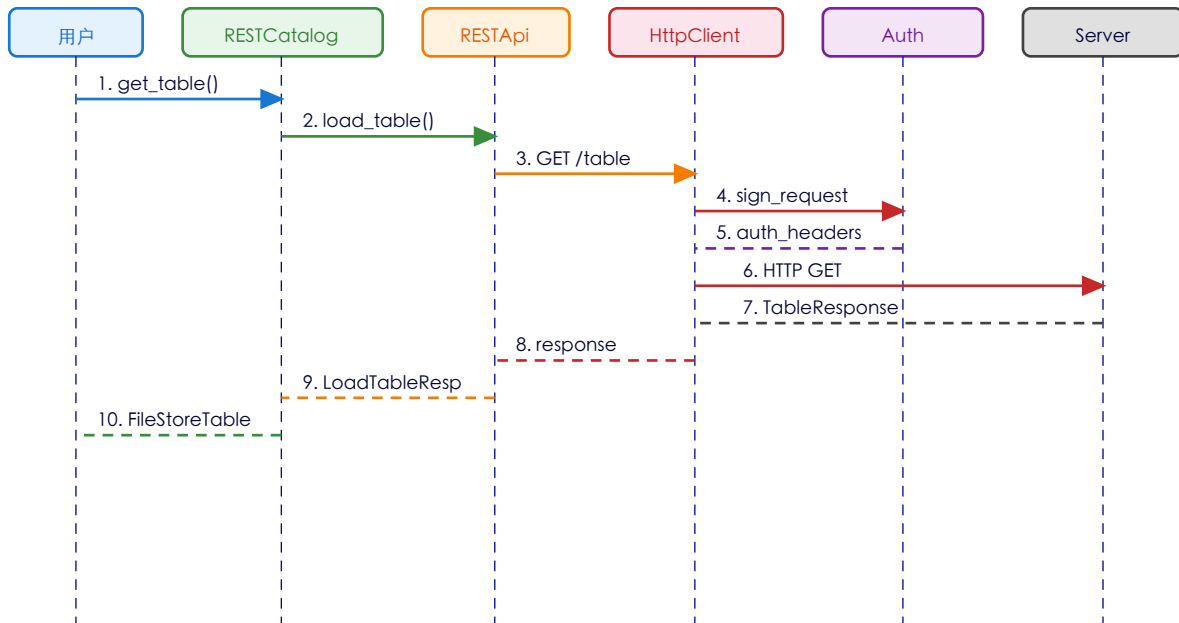


图5-3 REST Catalog 加载表时序图

第6章 关键场景调用链

6.1 场景一：创建 Catalog 并获取表

触发条件：用户通过 CatalogFactory.create() 创建 Catalog 并调用 get_table()

步骤	类.方法	说明
1	CatalogFactory.create(options)	根据 metastore 选择 Catalog 类型
2	FileSystemCatalog.__init__(options)	初始化仓库路径，创建 FileIO
3	FileIO.get(warehouse, options)	根据 scheme 选择 LocalFileIO/PyArrowFileIO
4	Catalog.get_table(identifier)	查询表 Schema
5	SchemaManager.latest()	读取最新 TableSchema
6	FileStoreTable.__init__(...)	构建表实例，解析主键/分区/BucketMode

6.2 场景二：主键表数据读取

触发条件：对主键表执行 read_builder → scan → read

步骤	类.方法	说明
1	ReadBuilder.with_filter(predicate)	设置过滤谓词
2	ReadBuilder.new_scan().plan()	创建 TableScan 执行扫描
3	SnapshotManager.get_latest_snapshot()	获取最新快照
4	ManifestListManager.read_all(snapshot)	读取 Manifest 列表
5	FileScanner.scan()	谓词过滤 + 生成 Split
6	MergeFileSplitRead.create_reader()	IntervalPartition → SortMerge 归并
7	KeyValueUnwrapRecordReader	去除 KV 包装
8	FilterRecordReader	应用用户谓词

6.3 场景三：数据写入与快照提交

触发条件：用户 write(PyArrow Table) 并 commit

步骤	类.方法	说明
1	BatchWriteBuilder.new_write()	创建 TableWrite
2	TableWrite.write(arrow_table)	数据分区 + Bucket 路由
3	RowKeyExtractor.extract(row)	提取分区键/主键/Bucket
4	FileWriter.write_batch(data)	写入数据文件
5	FileStoreCommit.commit(msgs, id)	开始提交
6	ManifestFileManager.write(entries)	写入 Manifest
7	SnapshotCommit.commit(snapshot)	原子提交(乐观锁)
8	CommitRollback (失败时)	冲突回滚 + 指数退避重试

6.4 场景四：数据演进读取

触发条件：表启用 data-evolution，读取跨 Schema 版本文件的 Split

步骤	类.方法	说明
1	DataEvolutionSplitRead.__init__()	检查 IndexedSplit 获取 row_ranges
2	_split_by_row_id(files)	按 firstRowId 分组文件
3	_split_field_bunches(files)	拆分 DataBunch + BlobBunch
4	_create_union_reader(files)	创建 DataEvolutionMergeReader
5	FormatReader (各格式)	按格式读取各字段
6	DataEvolutionMergeReader	按 offsets 拼接列

6.5 场景五：CLI 表数据查询

触发条件：执行 paimon table select 命令

步骤	类.方法	说明
1	cli_main.main()	解析参数，路由到 table
2	cli_table.select(args)	解析 --where/--columns/--limit
3	CatalogFactory.create(options)	创建 Catalog
4	table.new_read_builder()	构建 ReadBuilder
5	scan.plan() → read.to_arrow()	扫描读取
6	print_table(arrow_table, limit)	格式化输出

6.6 场景六：端到端建表写入读取

触发条件：创建新表 → 写入数据 → 读取验证

步骤	类.方法	说明
1	CatalogFactory.create(options)	创建 Catalog
2	catalog.create_database('db', True)	创建数据库
3	Schema(fields, pks, partition_keys)	构建 Schema
4	catalog.create_table(id, schema, False)	建表
5	table.new_batch_write_builder().new_write()	创建写入器
6	write.write(data) → commit()	写入并提交
7	read_builder.new_scan().plan()	扫描
8	read.to_arrow()	读取验证

第7章 配置与扩展

7.1 核心配置

配置项	默认值	说明
warehouse	(必填)	数据仓库根路径
metastore	filesystem	Catalog类型: filesystem/rest
bucket	1	Bucket数,-1 动态,-2延迟
file.format	parquet	文件格式: parquet/orc/avro/lance
snapshot.commit.retries	10	提交最大重试次数
read.batch-size	1024	每批读取行数
data-evolution.enabled	false	数据演进开关
row-tracking.enabled	false	行追踪开关
scan.tag-name	(无)	Time Travel 标签
branch	main	当前分支

7.2 扩展点

- Catalog 扩展：实现 Catalog ABC + CatalogFactory.CATALOG_REGISTRY 注册
- FileIO 扩展：实现 FileIO ABC + FileIO.get() scheme 路由
- 文件格式扩展：SplitRead.file_reader_supplier() 添加新格式分支
- 认证扩展：实现 AuthProvider ABC + AuthProviderFactory 注册
- 集成扩展：参考 integrations/ray/ 和 integrations/torch/

第8章 设计亮点与总结

8.1 设计亮点

- 纯 Python 实现，零 JVM 依赖：通过 PyArrow + fastavro 替代 Java 生态，降低部署复杂度
- 五层清晰架构：CLI → Catalog → Table → 读写引擎 → 基础设施，ABC 接口解耦
- 策略模式读取引擎：RawFile/MergeFile/DataEvolution 三种策略覆盖所有读取场景
- 乐观锁提交机制：指数退避 + 冲突检测 + 回滚，保证并发写入正确性
- 丰富格式支持：Parquet/ORC/Avro/Lance/Blob 五种，统一 RecordBatchReader 抽象
- 数据演进：跨 Schema 文件合并读取，FieldBunch 拆分 + offsets 映射
- 完善认证体系：BearerToken + DLF(VPC/公网双签名)，AuthProvider 可扩展
- Python 3.6 兼容：条件导入 + fastavro_py36_compat 兼容层

8.2 设计限制

- 写入格式主要支持 Parquet/ORC，JSON/CSV 不支持读取
- 不支持 Compaction，需依赖 Java 端
- 流式写入支持度有限，以批量写入为主

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	五层架构清晰，ABC 接口设计规范
设计模式运用	★★★★★	8种设计模式合理应用
可扩展性	★★★★☆	Catalog/FileIO/Auth/Format 均可扩展
测试覆盖	★★★★☆	80个测试文件覆盖核心功能
文档完整度	★★★☆☆	代码注释较少，依赖外部文档